

# Review — Group001\_EDA\_merged\_0905.ipynb

**Status: sections 4 and 5 have been APPLIED** (4 Sep, later the same day). The notebook now runs 13/13 clean with all fixes in and the eight figures re-exported to `review/figures_0905/`. Two further problems were found only after the fixes were drawn — Figure 6's intervals do **not** quite overlap (3.733 vs 3.730), and Figure 8's helpful-votes panel needed an axis floor once the bars came off — and both are fixed. Quote style and the coverage cell's long lines were deliberately left alone. **This file is kept as the audit trail of what was wrong and why.** For what to do next, read `Group001_handover_0905.md`.

Reviewed 4 September 2026 against the six standardised CSVs in `00_Master/outputs/`. All 13 code cells were re-executed from a fresh interpreter with `matplotlib` in non-interactive mode.

## 1. Run result

- **13 / 13 code cells ran clean, ~2.4 s, no errors and no warnings.**
- Every printed value reproduced the output already stored in the notebook, character for character. Nobody has a stale cell.
- The six CSV fingerprints printed by Section 0 match across all figures, so all eight figures were built on one copy of the data.
- Coverage cell passes on its own assertions: 8 figures, 6/6 categories, 6/6 tables, 5 relational.

## 2. Every number in the prose was re-derived independently

I recomputed each quantity a figure's markdown claims, from the CSVs, without using the notebook's own code. **All of them are correct**, including the four that no cell prints:

| Claim                                                                   | Where | Recomputed                    |
|-------------------------------------------------------------------------|-------|-------------------------------|
| mean is 19% above median; mean exceeds 59% of orders; skew 1.38         | Fig 1 | 1.192x, 59.0%, 1.38           |
| 3.53x inflation; \$16.47m gross vs \$14.92m invoiced; 20,154 units      | Fig 2 | exact                         |
| CV 6.1 / 4.2 / 4.0; 384–443 orders; 76 deliveries fall in Jan 2019      | Fig 3 | exact; 76 confirmed           |
| cells span 81.8%–95.8%; Express 89.7% vs Standard 89.4%; Express is 18% | Fig 4 | exact                         |
| <b>promised_days 5.03 Express vs 4.99 Standard</b>                      | Fig 4 | 5.034 / 4.993                 |
| 271 non-Latin at 607 chars vs 6,729 Latin at 927; lengths 180–2,492     | Fig 5 | exact                         |
| <b>+0.12 stars, 2.5 standard errors</b>                                 | Fig 6 | +0.120, <b>2.55 SE</b>        |
| <b>delivery_experience agrees on all 7,000 rows</b>                     | Fig 6 | 707/6,293, zero disagreements |

| Claim                                                                               | Where | Recomputed                                               |
|-------------------------------------------------------------------------------------|-------|----------------------------------------------------------|
| 1.5 SE and 2.4 SE segment gaps; all 500 customers have an order                     | Fig 7 | 1.48 / 2.36 / 2.44 SE; 0 customers without orders        |
| quintile ratings 3.879 → 3.548 → 3.684; helpful votes 44.0–45.5                     | Fig 8 | exact                                                    |
| Finding 10: <b>delivery_note_clean</b> is the same partition as the outcome columns | §3    | 4,472 / 528 against all four columns, zero disagreements |

The narratives are honest. Nothing is inflated and both nulls are correctly refused.

### 3. Blocking — the notebook cannot go to Jasmine yet

Task 6 is written *from* this notebook. These are empty, so there is nothing to write from:

| Section                                   | Owner   | State                                                       |
|-------------------------------------------|---------|-------------------------------------------------------------|
| §1 Context and data-preparation assurance | Yandu   | still a brief listing candidates, not prose                 |
| Findings 3, 4                             | Jasmine | literally <b>Replace</b> .                                  |
| Findings 5, 6                             | Yandu   | literally <b>Replace</b> .                                  |
| Findings 7, 8                             | Shawn   | literally <b>Replace</b> .                                  |
| Findings 9, 10                            | Yandu   | half-written, with bracketed instructions still in the text |
| MLQ-1 evidence + business decision        | Jasmine | <b>*Replace*</b>                                            |
| MLQ-3 evidence + business decision        | Shawn   | <b>*Replace*</b>                                            |
| MLQ-4 evidence + business decision        | Yandu   | <b>*Replace*</b>                                            |
| §5 Limitations and conclusion             | Jasmine | still a brief                                               |
| References                                | —       | empty                                                       |

**6 of 10 findings and 3 of 5 MLQ rows are unwritten.** Everything below is small by comparison; this is the gate.

A free finding, already verified

One of the six empty finding slots can be filled today with no new analysis.

**orders.expedited\_delivery** is **deliveries.service\_level** re-encoded — 901 True / 901 Express, 4,099 / 4,099, zero exceptions. With **delivery\_note\_clean** (Finding 10) and **product\_reviews.delivery\_experience** (Figure 6's limitation) that makes **three** redundant column pairs across two source systems. Three is a source-system finding in its own right and a standing leakage warning for every MLQ.

## 4. Defects to fix — tested paste-in replacements

### 4.1 Section 5 says "three columns have no variance". There are seven. (*Jasmine*)

`order_status`, `delivery_status` and `verified_purchase` are named. Also single-valued: `orders.currency`, `products.tax_category`, `customers.home_state`, `customers.home_country`. A marker can check this in one line, so fix the count.

### 4.2 Figure 3's "How to build it" does not describe how Figure 3 was built. (*Jasmine*)

It currently holds a note about `delivered_date` running into January 2019 and 76 false range failures. That is a data-preparation point — it belongs in §1 or as a limitation. A reader of Figure 3 is never told the figure is three stacked panels of revenue, count and mean value over the twelve months. Replace with:

**How to build it:** Group the 5,000 orders by month of `order_timestamp` and draw three stacked panels sharing one x-axis — total revenue, order count, and mean order value. The top panel is the product of the other two, so stacking them shows which component moves when revenue moves. Each panel starts at zero, which is what makes the flatness of the series visible rather than magnified by a cropped axis.

Also: revenue runs to AUD **1.36m**, not 1.35m (1,355,379). Jasmine will copy this number into the report, so fix it at source.

### 4.3 Figures 2 and 6 have no figure number in the chart title. (*Echo*)

Figures 1, 3, 4, 7 begin **Figure N –**. Figure 5 and 8 use **Figure N.**. Figures 2 and 6 carry no number at all. Jasmine is going to crop eight PNGs into a report and reference them by number. Pick one form — **Figure N –** is the majority — and use it eight times.

Figure 2, line 43:

```
fig.suptitle('Figure 2 – Category revenue rank versus unit-sales rank\n'
             f'{len(lines):,} order lines, one row per order line')
```

### 4.4 Zero-baseline bars hide the finding in Figures 6 and 8. (*Echo, Shawn*)

Both draw means on a 0–5 star axis. Figure 6's whole finding is a 0.12-star gap and Figure 8's is a 0.33-star swing; on a 0–5 axis both are invisible, so the chart shows nothing and the reader has to take the caption on trust. Figure 4 already solves exactly this with a dot-and-interval and a Wilson interval. Making 6 and 8 match 4 does three things at once: the nulls become *visible* as overlapping intervals, the four unprinted numbers get printed, and the eight figures start looking like one team's work.

### Figure 6 — tested replacement for the whole cell (*Echo*):

```
# --- Figure 6 (Echo) ---
f6 = product_reviews[['order_id', 'rating']].merge(
    deliveries[['order_id', 'on_time_in_full']], on='order_id', how='left')
at_grain(f6, len(product_reviews), 'reviews + deliveries')
```

```

f6['rating'] = num(f6['rating'])
f6['delivery'] = f6['on_time_in_full'].astype(str).map({'True': 'On time',
'False': 'Late'})

# Mean, and the uncertainty around it, so a 707-review group is visibly
less
# certain than a 6,293-review one.
g6 = f6.groupby('delivery')['rating'].agg(['mean', 'std', 'size']).loc[['On
time', 'Late']]
g6['se'] = g6['std'] / g6['size'] ** .5
print(g6.round(3).to_string())

# The interpretation quotes this ratio, so the cell prints it rather than
asserting it.
diff = g6.loc['Late', 'mean'] - g6.loc['On time', 'mean']
se_diff = (g6['se'] ** 2).sum() ** .5
print(f'\nLate minus On time: {diff:+.3f} stars = {diff / se_diff:.2f}
standard errors')

# The review table carries its own lateness field. If it never disagrees
with the
# delivery table, the review text is not an independent record of lateness.
chk = product_reviews[['order_id', 'delivery_experience']].merge(
    deliveries[['order_id', 'on_time_in_full']], on='order_id', how='left')
agree = (chk['delivery_experience'].eq('on_time')
        == chk['on_time_in_full'].astype(str).eq('True')).mean()
print(f"the reviews' own delivery_experience agrees with on_time_in_full on
"
      f'{agree:.1%} of {len(chk):,} reviews')

fig, ax = plt.subplots(figsize=(7.5, 3.2))
y = range(len(g6))
ax.errorbar(g6['mean'], y, xerr=1.96 * g6['se'], fmt='o', color='tab:blue',
capsize=4)
for i, r in enumerate(g6.itertuples()):
    ax.text(r.mean, i + .22, f'{r.mean:.2f}    n = {r.size:,}', ha='center',
fontSize=9)
ax.set_yticks(list(y))
ax.set_yticklabels(g6.index)
ax.set_ylim(-.6, len(g6) - .3)
ax.set_xlim(3.5, 4.0)
ax.set_xlabel('Mean rating on the 1-5 star scale, with 95% confidence
interval')
ax.set_title('Figure 6 – Mean rating by delivery outcome\n'
            f'{len(f6):,} reviews, one row per review; the two intervals
overlap')
ax.grid(axis='x', alpha=.3)
plt.tight_layout()
plt.show()

```

Verified output: On time 3.699 (se 0.016, n 6,293), Late 3.819 (se 0.044, n 707),  
+0.120 stars = 2.55 standard errors, agrees on 100.0% of 7,000 reviews. The cropped

x-axis is fine here and is the same choice Figure 4 makes: the interval, not the baseline, is what stops the reader over-reading the gap, and the axis label names the 1–5 scale.

**Figure 8 — tested replacement for the plotting block (Shawn).** Add `sd_rating` to the aggregation, then swap the two bar panels:

```
mean_rating=("rating", "mean"),
sd_rating=("rating", "std"),
median_helpful_votes=("helpful_votes", "median"),
)
.reset_index(drop=True)
)
summary8["se"] = summary8["sd_rating"] / summary8["n"] ** .5
print(summary8.round(3).to_string(index=False))
```

```
axes[0].errorbar(x, summary8["mean_rating"], yerr=1.96 * summary8["se"],
                 fmt="o", color="#4C72B0", capsize=4)
axes[0].set_title("Mean rating, with 95% confidence interval")
axes[0].set_ylabel("Mean rating on the 1–5 star scale")

axes[1].plot(x, summary8["median_helpful_votes"], "o", color="#C4622D")
axes[1].set_title("Median helpful votes")
axes[1].set_ylabel("Median helpful votes per review")

for ax in axes:
    ax.set_xticks(list(x))
    ax.set_xticklabels(labels)
    ax.set_xlabel("Order-item unit-price quintile (AUD)")
    ax.grid(axis="y", alpha=.3)

fig.suptitle("Figure 8 – Rating and helpful votes by unit-price quintile\n"
             f"{len(f8):,} reviewed order items, one row per review")
```

#### 4.5 Figure 8's narrative undersells a real result. (Shawn)

This is the one place where the prose is *too* cautious. Standard errors are ~0.033 per quintile, so the drop from Q1 (3.879) to Q4 (3.548) is **about 7 standard errors** — not noise. What is not monotonic is the rebound in Q5 (3.684). "No clear monotonic price relationship" is technically true but reads as a null, and it is not one. Suggested:

Mean rating falls steadily across the first four price quintiles, from 3.88 to 3.55 — a 0.33-star drop that is roughly seven standard errors and far larger than the spread of the intervals — then rebounds to 3.68 in the top quintile. The relationship is real but not monotonic, so it cannot be summarised as "dearer products rate worse". Median helpful votes stay between 44.0 and 45.5 across all five quintiles and carry no signal.

#### 4.6 Figure 5's main claim has no numbers behind it. (Shawn)

The cell prints only n and median length per script group. The rating-versus-length half of the question — the reason there are ten boxplots — is never quantified, and the claim "neither group showed a clear monotonic rating-length pattern" is only half right. Add after `print(summary5...)`:

```
by_rating = f5.pivot_table(index="rating", columns="script_group",
                           values="review_length_chars", aggfunc="median")
print("\nmedian cleaned length (characters) by rating")
print(by_rating.round(0).to_string())
print(f"\nlengths run {f5['review_length_chars'].min():,.0f} to "
      f"{f5['review_length_chars'].max():,.0f} characters")
```

Verified output:

| script_group | Contains non-Latin | Latin only |
|--------------|--------------------|------------|
| rating       |                    |            |
| 1            | 746.0              | 955.0      |
| 2            | 659.0              | 980.0      |
| 3            | 513.0              | 928.0      |
| 4            | 659.0              | 919.0      |
| 5            | 498.0              | 907.0      |

Latin-only reviews shrink about 7% from rating 2 to rating 5 — a weak but orderly decline, not "no pattern". The non-Latin column really is noise (n = 271 split five ways). The narrative should say those two different things separately.

#### 4.7 Figure 4 — print the two numbers the prose asserts. (*Jasmine*)

`promised_days` 5.03 vs 4.99 and the cell sizes are both quoted in the markdown but never printed. Append to the cell:

```
print('\ndeliveries per cell: Express %d-%d, Standard %d-%d'
      % (g[g.service_level == 'Express']['n'].min(), g[g.service_level ==
'Express']['n'].max(),
        g[g.service_level == 'Standard']['n'].min(), g[g.service_level ==
'Standard']['n'].max()))
print('mean promised_days by service level')
print(num(deliveries['promised_days']).groupby(deliveries['service_level'])
      .mean().round(2).to_string())
```

Verified: Express 36–100, Standard 151–470; Express 5.03, Standard 4.99.

One wording point. "Every large swing sits in an Express cell of 36 to 100 deliveries" is true but circular — every Express cell is 36 to 100. Say it the way that actually carries the argument: "No Express cell holds more than 100 deliveries, so no cell in this design can resolve a gap of the size the chart appears to show."

Also, the chart title says "95% binomial" while the markdown says "Wilson". Same interval; use one name in both places.

---

## 5. Consistency — cheap, and it is what "everyone can read everyone's work" means

1. **Three names for the same header field.** Figures 1, 3, 4, 7 use *How to build it*; Figures 5 and 8 use *Method*; Figures 2 and 6 have neither. The last field is *Interpretation and limitation* in six figures and *Result and limitation* in Figures 2 and 6. Pick one set of five labels and use it eight times — a marker reading the header block should not have to work out that *Method* and *How to build it* are the same slot.
  2. **Quote style splits by author.** Figures 5 and 8 are 100% double-quoted with wrapped call arguments; the other six cells are 100% single-quoted and compact. It is obvious at a glance that two people wrote this. Single quotes are the majority; converting the two cells is a five-minute find-and-replace.
  3. **Fourteen code lines exceed the 98-character house rule** — cell 14 (2), cell 24 (2), cell 26 (1), cell 28 (9). The nine in the coverage cell are deliberate column alignment and are worth keeping; the other five are accidental.
  4. **Cell 7 does two things and the markdown above it describes one.** It defines `at_grain` and `num`, then prints a per-table primary-key uniqueness check. Section 0.1's markdown never mentions that check. One sentence fixes it.
  5. **Owner tags.** `**Owner:** Yandu` in the markdown and `# --- Figure 4 (Jasmine) ---` in the code are useful to the four of you and slightly odd in a submitted artefact. Either is defensible — decide once and apply it the same way everywhere, rather than leaving the current mix.
- 

## 6. Things that are already good, and should not be touched

- `at_grain()` on every join, with the expected row count named at the call site, is the strongest thing in the notebook. Figure 2's deliberate demonstration of the 3.53x inflation is the clearest teaching moment in it.
  - Figure 4's dot-and-interval is the right chart for a null and the right model for Figures 6 and 8.
  - Figure 7 is the only figure whose narrative can be re-derived entirely from its own printed output — the standard errors are printed, so the "1.5 SE" and "2.4 SE" claims are checkable without leaving the cell. That is the standard the others should meet.
  - Figure 8's assertion that the order-item unit price matches the catalogue price within 0.01 is a genuine cross-table integrity check, not decoration.
  - Both nulls (Figures 4 and 6) are refused correctly and neither is dressed up. The temptation to write "late deliveries lower ratings" was resisted.
- 

## 7. Suggested order of work

1. Yandu, Jasmine, Shawn write the six findings and three MLQ rows — §3 is the gate.
2. Yandu writes §1 as prose; Jasmine writes §5 and fixes the column count.
3. Owners apply 4.2 through 4.7 (all tested above, ~30 minutes total).
4. Someone does the §5 consistency pass in one sitting so it stays consistent.

5. One person runs the notebook top to bottom, confirms all 13 cells and the coverage assertions still pass, and only then hands it to Jasmine for the report.